

23CSE314 Compiler Design

Labsheet1

Sample:

```
C:\Users\exam\23219>java -cp .;antlr-4.13.2-complete.jar org.antlr.v4.gui.TestRig SimpleLexer tokens -tokens
if a+10
^Z
[@0,0:1='if',<KEYWORD>,1:0]
[@1,3:3='a',<ID>,1:3]
[@2,4:4='+',<OP>,1:4]
[@3,5:6='10',<INT>,1:5]
[@4,9:8='<EOF>',<EOF>,2:0]
```

Exercises

1. Test the lexer for the following inputs and record the observations

a. x-5+3

```
C:\Users\exam\23219>java -cp .;antlr-4.13.2-complete.jar org.antlr.v4.gui.
TestRig SimpleLexer tokens -tokens
x-5+3
^Z
[@0,0:0='x',<ID>,1:0]
[@1,1:1='-','<OP>,1:1]
[@2,2:2='5',<INT>,1:2]
[@3,3:3='+',<OP>,1:3]
[@4,4:4='3',<INT>,1:4]
[@5,7:6='<EOF>',<EOF>,2:0]
```

b. total_sum+value2

```
C:\Users\exam\23219>java -cp .;antlr-4.13.2-complete.jar org.antlr.v4.gui.
TestRig SimpleLexer tokens -tokens
total_sum+value2
^Z
line 1:5 token recognition error at: '_'
[@0,0:4='total',<ID>,1:0]
[@1,6:8='sum',<ID>,1:6]
[@2,9:9='+',<OP>,1:9]
[@3,10:15='value2',<ID>,1:10]
[@4,18:17='<EOF>',<EOF>,2:0]
```

c. ifelse

```
C:\Users\exam\23219>java -cp .;antlr-4.13.2-complete.jar org.antlr.v4.gui.
TestRig SimpleLexer tokens -tokens
ifelse
^Z
[@0,0:5='ifelse',<ID>,1:0]
[@1,8:7='<EOF>',<EOF>,2:0]
```

d. while while1

```
C:\Users\exam\23219>java -cp .;antlr-4.13.2-complete.jar org.antlr.v4.gui.  
TestRig SimpleLexer tokens -tokens  
while while1  
^Z  
[@0,0:4='while',<KEYWORD>,1:0]  
[@1,6:11='while1',<ID>,1:6]  
[@2,14:13='<EOF>',<EOF>,2:0]
```

e. if a + 10

```
C:\Users\exam\23219>java -cp .;antlr-4.13.2-complete.jar org.antlr.v4.gui.  
TestRig SimpleLexer tokens -tokens  
if a + 10  
^Z  
[@0,0:1='if',<KEYWORD>,1:0]  
[@1,7:7='a',<ID>,1:7]  
[@2,11:11='+',<OP>,1:11]  
[@3,14:15='10',<INT>,1:14]  
[@4,18:17='<EOF>',<EOF>,2:0]
```

2. Modify the grammar by

a. Interchange the order of the IF and ID rules and run the lexer with the input 'if' and record the token generated

```
C:\Users\exam\23219>java -cp .;antlr-4.13.2-complete.jar org.antlr.v4.gui.  
TestRig SimpleLexer tokens -tokens  
if  
^Z  
[@0,0:1='if',<ID>,1:0]  
[@1,4:3='<EOF>',<EOF>,2:0]
```

b. Restore the correct order: IF : 'if' ; ID : [a-z][a-z0-9]* ; Test the input: 'if if1' and record the tokens generated.

```
C:\Users\exam\23219>java -cp .;antlr-4.13.2-complete.jar org.antlr.v4.gui.  
TestRig SimpleLexer tokens -tokens  
if if1  
^Z  
[@0,0:1='if',<'if'>,1:0]  
[@1,3:5='if1',<ID>,1:3]  
[@2,8:7='<EOF>',<EOF>,2:0]
```

c. Add the following rules: EQ : '=='; ASSIGN : '=' ; Test the lexer with a==b and record the tokens generated

```
C:\Users\exam\23219>java -cp .;antlr-4.13.2-complete.jar org.antlr.v4.gui.
TestRig SimpleLexer tokens -tokens
a==b
^Z
[@0,0:0='a',<ID>,1:0]
[@1,1:2=='==',<'=='>,1:1]
[@2,3:3='b',<ID>,1:3]
[@3,6:5='<EOF>',<EOF>,2:0]
```

d. Interchange the order of the rules '==' and '=' and record the change in the tokens generated.

```
C:\Users\exam\23219>java -cp .;antlr-4.13.2-complete.jar org.antlr.v4.gui.
TestRig SimpleLexer tokens -tokens
a==b
^Z
[@0,0:0='a',<ID>,1:0]
[@1,1:2=='==',<'=='>,1:1]
[@2,3:3='b',<ID>,1:3]
[@3,6:5='<EOF>',<EOF>,2:0]
```

3. Modify the grammar to include the following and test with various inputs

i. for

```
C:\Users\exam\23219>java -cp .;antlr-4.13.2-complete.jar org.antlr.v4.gui.
TestRig SimpleLexer tokens -tokens
for a in range()
^Z
line 1:14 token recognition error at: '('
line 1:15 token recognition error at: ')'
[@0,0:2='for',<KEYWORD>,1:0]
[@1,4:4='a',<ID>,1:4]
[@2,6:7='in',<ID>,1:6]
[@3,9:13='range',<ID>,1:9]
[@4,18:17='<EOF>',<EOF>,2:0]
```

ii. relational operators

```
C:\Users\exam\23219>java -cp .;antlr-4.13.2-complete.jar org.antlr.v4.gui.
TestRig SimpleLexer tokens -tokens
2>3
^Z
[@0,0:0='2',<INT>,1:0]
[@1,1:1='>',<RELATIONALOP>,1:1]
[@2,2:2='3',<INT>,1:2]
[@3,5:4='<EOF>',<EOF>,2:0]
```

iii. Floating point numbers

```
C:\Users\exam\23219>java -cp .;antlr-4.13.2-complete.jar org.antlr.v4.gui.
TestRig SimpleLexer tokens -tokens
2.0
^Z
[@0,0:2='2.0',<FLOAT>,1:0]
[@1,5:4='<EOF>',<EOF>,2:0]
```

iv. Single line and multiline comments (should be ignored)

```
C:\Users\exam\23219>java -cp .;antlr-4.13.2-complete.jar org.antlr.v4.gui.
TestRig SimpleLexer tokens -tokens
//this is a single line comment
/* this is multi
line
comment*/
^Z
[@0,68:67='<EOF>',<EOF>,5:0]
```

v. Modify the ID rule to allow underscore (_) at the beginning and within identifiers.

```
C:\Users\exam\23219>java -cp .;antlr-4.13.2-complete.jar org.antlr.v4.gui.
TestRig SimpleLexer tokens -tokens
int _a=2;
^Z
line 1:8 token recognition error at: ';'
[@0,0:2='int',<ID>,1:0]
[@1,4:5='_a',<ID>,1:4]
[@2,6:6='=',<'='>,1:6]
[@3,7:7='2',<INT>,1:7]
[@4,11:10='<EOF>',<EOF>,2:0]
```

4.i. To add a lexer rule to catch invalid characters and report an error.

```
C:\Users\exam\23219>java -cp .;antlr-4.13.2-complete.jar org.antlr.v4.gui.
TestRig SimpleLexer tokens -tokens
int x=100$;
^Z
Invalid character: $
Invalid character: ;
[@0,0:2='int',<ID>,1:0]
[@1,4:4='x',<ID>,1:4]
[@2,5:5='=',<'='>,1:5]
[@3,6:8='100',<INT>,1:6]
[@4,9:9='$',<INVALID>,1:9]
[@5,10:10=';',<INVALID>,1:10]
[@6,13:12='<EOF>',<EOF>,2:0]
```

ii. To recognize Data types (int, float, char), and Semicolon (;)

```
C:\Users\exam\23219>java -cp .;antlr-4.13.2-complete.jar org.antlr.v4.gui.
TestRig SimpleLexer tokens -tokens
int x=100;
float z=$2;
^Z
Invalid character: $
[@0,0:2='int',<DATATYPE>,1:0]
[@1,4:4='x',<ID>,1:4]
[@2,5:5='=',<'='>,1:5]
[@3,6:8='100',<INT>,1:6]
[@4,9:9=';',<'>',1:9]
[@5,12:16='float',<DATATYPE>,2:0]
[@6,18:18='z',<ID>,2:6]
[@7,19:19='=',<'='>,2:7]
[@8,20:20='$',<INVALID>,2:8]
[@9,21:21='2',<INT>,2:9]
[@10,22:22=';',<'>',2:10]
[@11,25:24='<EOF>',<EOF>,3:0]
```